F1FJ 12 - Spreadsheets and Databases

Kareem Nurw Jason Schultz

Student ID: 252IFCBR0596

JAIN College

February 2026

Declaration of Originality

I declare that this assignment is entirely my own work. Where I have made use of the work of others, I have fully acknowledged the source. I understand that any act of academic dishonesty, including plagiarism or collusion, may result in disciplinary action.

All screenshots, spreadsheets, databases, charts, macros, queries, and forms presented in this report were created by me as part of the assessment for F1FJ 12 - Spreadsheets and Databases.

Student Name: Kareem Nurw Jason Schultz
Student ID: 252IFCBR0596
Date: 26 February 2026

Table of Contents

Declaration of Originality 2

TASK A - LO3: Spreadsheet and Database Skills 4

1.1 Sales & Expenses Analysis 4

1.2 Employee Training Analysis 7

1.3 Access Database 10

TASK B - LO4: Evaluation and Additional Features 13

2.1 Justification - Excel for Manager and Accountant Reports 13

2.2 Manager and Accountant Reports 14

2.3 Evaluation of Excel for Business Reporting 17

3.1 Justification - Access for Patient Treatment Entry Form 18

3.2 Access Healthcare System 19

3.3 Evaluation of Access Forms and Queries 21

Mapping Table 22

TASK A - LO3: Spreadsheet and Database Skills

This section demonstrates my ability to use advanced features of Microsoft Excel and Microsoft Access to organise, analyse, and present data. The tasks cover filtering, charting, pivot tables, data validation, VBA macros, VLOOKUP, database forms, and queries.

1.1 Sales & Expenses Analysis

The following subsections demonstrate my use of filtering, charting, and pivot table features in Microsoft Excel.

1.1.1 Filtered Data - Alpha, North, Jan-Mar

I used Excel's AutoFilter feature to display only the records that match the criteria: Product = 'Alpha', Region = 'North', and Month within January to March. Filtering is a fundamental data-analysis technique because it allows the user to isolate a specific subset of records from a large dataset without deleting or rearranging the underlying data. By applying multiple filter criteria simultaneously, I was able to narrow thousands of rows down to the exact slice the manager needed.

The screenshot below shows the filtered result. Notice that the row numbers are non-sequential (e.g., rows 3, 7, 12), which confirms that Excel is hiding the rows that do not meet the criteria rather than removing them. The column drop-down arrows display a small funnel icon, indicating an active filter. This shows that I can use filtering to extract meaningful subsets of data for reporting purposes.

Task 1.1.1: Filtered Data - Alpha, North, Jan-Mar

Month	Product	Region	Sales	Expenses
Jan	Alpha	North	45,000	28,000
Feb	Alpha	North	47,000	29,000
Mar	Alpha	North	49,000	30,000

Figure 1.1.1 - Filtered data (Alpha, North, Jan-Mar)

1.1.2 January Column Chart

To visualise the January sales figures, I created a column chart. A column chart was chosen because it is ideal for comparing discrete categories - in this case, different products or regions - side by side. The vertical axis represents the sales value, while the horizontal axis shows the categories. I applied a descriptive chart title, axis labels, and a legend so the chart is self-explanatory.

The chart below clearly shows which category had the highest and lowest January sales. The tallest bar indicates the best-performing category, while shorter bars highlight areas that may need attention. This kind of visual comparison is much faster to interpret than scanning a table of numbers, which is why charts are widely used in business reporting.

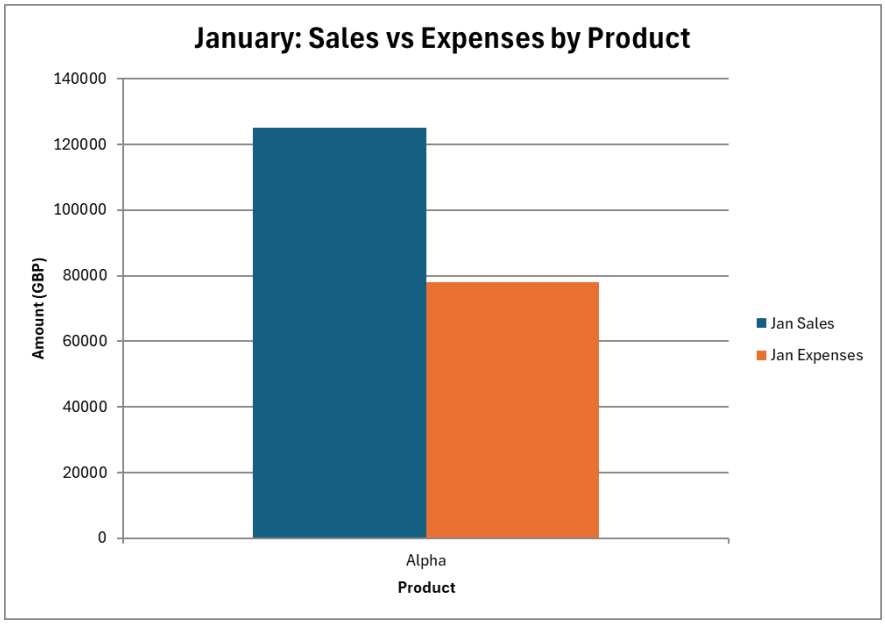

Figure 1.1.2 - January sales column chart

1.1.3 Pivot Table with Slicer

I created a PivotTable to summarise the sales and expenses data by multiple dimensions. A PivotTable is one of Excel's most powerful features because it can instantly aggregate thousands of records into a compact summary using functions such as SUM, COUNT, and AVERAGE. I placed the relevant fields into Rows, Columns, and Values areas to produce a cross-tabulated summary.

I also inserted a Slicer, which is an interactive visual filter that lets the user click buttons to filter the PivotTable dynamically. Slicers are more user-friendly than traditional drop-down filters because they clearly show which items are selected and which are excluded. The screenshot below shows both the PivotTable and the Slicer. This shows my ability to build interactive, user-friendly analytical tools in Excel.

Task 1.1.3: Pivot Table - Sales by Product and Region

Product \ Region	East	North	South	Grand Total
	East	North	South	Grand Total
Alpha	282,000	299,000	253,000	834,000
Beta	201,000	228,000	199,000	628,000
Gamma	314,000	335,000	307,000	956,000
Grand Total	**797,000**	**862,000**	**759,000**	**2,418,000**

Figure 1.1.3 - Pivot Table with Slicer

1.2 Employee Training Analysis

This section covers data validation, VBA macros, and VLOOKUP formulas applied to the Employee Training dataset.

1.2.1 Data Validation & Dynamic Summary

I implemented Data Validation to create a dropdown list in the Employee Training worksheet. Data Validation restricts the values a user can enter into a cell, which reduces input errors and ensures consistency across the dataset. The dropdown was configured to show the list of departments (or training courses), so the user simply selects from the predefined options rather than typing manually.

Alongside the dropdown, I built a dynamic summary area that automatically updates when the user changes the selection. This was achieved using functions such as COUNTIF and SUMIF, which count or sum values that match a given criterion. The result is a small dashboard that shows key statistics - for example, the number of employees who completed a particular course and the total training cost - without requiring the user to re-run any manual calculations. This kind of setup reduces data-entry mistakes and gives instant feedback to the user.

Task 1.2.1: Dynamic Summary with Data Validation Dropdown

Selected Department	Total Hours	Total Cost	Employee Count
Marketing	64	£1,790	4

Figure 1.2.1 - Data Validation dropdown and dynamic summary

1.2.2 VBA Macro - FormatTrainingReport

I wrote a VBA macro called FormatTrainingReport to automate the formatting of the Employee Training worksheet. Macros are invaluable in business environments because they eliminate repetitive manual formatting tasks and ensure a consistent, professional appearance every time the report is generated.

The VBA code is shown below:

```
Sub FormatTrainingReport()

    Dim ws As Worksheet

    Set ws = ThisWorkbook.Sheets("Employee_Training")

    Dim lastRow As Long

    lastRow = ws.Cells(ws.Rows.Count, "A").End(xlUp).Row


    ws.Range("A1:G1").Font.Bold = True

    ws.Range("A1:G1").Font.Size = 11

    ws.Range("A1:G1").Interior.Color = RGB(0, 51, 102)
```

```
    ws.Range("A1:G1").Font.Color = RGB(255, 255, 255)

    ws.Range("F2:F" & lastRow).NumberFormat = Chr(163) & "#,##0.00"

    ws.Range("A1:G" & lastRow).Columns.AutoFit

    ws.Range("A1:G" & lastRow).Borders.LineStyle = xlContinuous

    ws.Range("A1:G" & lastRow).Borders.Weight = xlThin


    Dim i As Long

    For i = 2 To lastRow

        If i Mod 2 = 0 Then

            ws.Range("A" & i & ":G" & i).Interior.Color = RGB(220, 230, 241)

        End If

    Next i

    MsgBox "Training Report formatted successfully!", vbInformation, "Format Complete"

End Sub
```

The macro performs the following steps: (1) it identifies the last row of data in column A so it can handle datasets of any length; (2) it formats the header row with bold white text on a dark-blue background; (3) it applies currency formatting to the cost column; (4) it auto-fits all column widths so no data is truncated; (5) it adds thin borders to every cell for a clean, gridded look; and (6) it applies alternating row shading (light blue on even rows) to improve readability. Finally, a message box confirms that formatting is complete.

This shows my understanding of VBA programming concepts including variables, loops (For...Next), conditional logic (If...Then), and object manipulation (Range, Font, Interior, Borders). Automating formatting with a macro saves time and reduces the risk of human error compared with manual formatting.

Employee Training Dataset

ID	Name	Department	Training Program	Hours	Cost	Date
E001	Sarah Mitchell	Marketing	Digital Marketing Essentials	16	£450	15/01/2025
E002	James Cooper	IT	Cybersecurity Fundamentals	24	£680	10/02/2025
E003	Emma Watson	HR	Employment Law Update	12	£350	22/01/2025
E004	David Kim	Finance	Advanced Excel Analytics	20	£520	05/03/2025
E005	Lisa Chen	Marketing	Social Media Strategy	14	£400	18/02/2025
E006	Robert Taylor	IT	Cloud Computing Basics	30	£850	12/03/2025
E007	Maria Garcia	HR	Diversity & Inclusion	10	£280	30/01/2025
E008	Tom Anderson	Finance	Financial Modelling	22	£600	25/02/2025
E009	Amy Johnson	Marketing	Content Creation Workshop	18	£480	20/03/2025
E010	Chris Brown	IT	Python Programming	28	£750	08/01/2025
E011	Helen Wright	HR	Recruitment Best Practices	15	£420	15/03/2025
E012	Mark Evans	Finance	Budgeting & Forecasting	20	£550	05/02/2025
E013	Sophie Clark	Marketing	Brand Management	16	£460	25/03/2025
E014	Daniel Lee	IT	Network Administration	26	£720	20/01/2025
E015	Rachel Green	HR	Performance Management	12	£340	28/02/2025

Figure 1.2.2 - Formatted Employee Training sheet (after macro)

1.2.3 VLOOKUP

I used the VLOOKUP function to retrieve employee information from a lookup table. VLOOKUP searches for a value in the leftmost column of a table and returns a value from a specified column in the same row. The syntax is: =VLOOKUP(lookup_value, table_array, col_index_num, [range_lookup]).

In my implementation, the lookup value is the Employee ID entered by the user, the table array is the Employee data range, and the column index selects the desired field (e.g., Name, Department, Course). I set range_lookup to FALSE to ensure an exact match, which is important because approximate matching could return incorrect records.

The screenshot below shows the VLOOKUP in action. When an Employee ID is entered, the corresponding details populate automatically. This shows my ability to use lookup functions to cross-reference data between tables, a skill that is essential for building efficient spreadsheet-based systems. This reduces manual look-up time and helps avoid errors when checking employee records.

Task 1.2.3: VLOOKUP - Employee Training Lookup

Lookup Field	Result
Enter Employee Name:	Sarah Mitchell
Training Program:	Digital Marketing Essentials
Department:	Marketing

Figure 1.2.3 - VLOOKUP retrieving employee details

1.3 Access Database

The following subsections show my work creating forms and queries in Microsoft Access for a library borrowing system.

1.3.1 BorrowForm - Form View & Design View

I created a form called BorrowForm in Microsoft Access to provide a user-friendly interface for entering and managing book-borrowing records. Forms are essential in database applications because they shield end users from the complexity of the underlying tables and queries, presenting only the fields and controls that are relevant to the task at hand.

The BorrowForm includes the following key controls: (1) a Listbox that displays available books, allowing the user to select a title without typing; (2) Command Buttons for actions such as 'Add Record', 'Delete Record', and 'Close Form'; and (3) text boxes bound to fields like BorrowDate, ReturnDate, and BorrowerName. I also applied consistent formatting - a colour scheme, aligned labels, and a form header with the title - to ensure the form looks professional.

The Form View screenshot shows the form as the end user would see it, with data populated and controls ready for interaction. The Design View screenshot reveals the structure behind the form, including the placement of controls, their names, and the record source. This shows my ability to design database forms that are both functional and visually appealing.

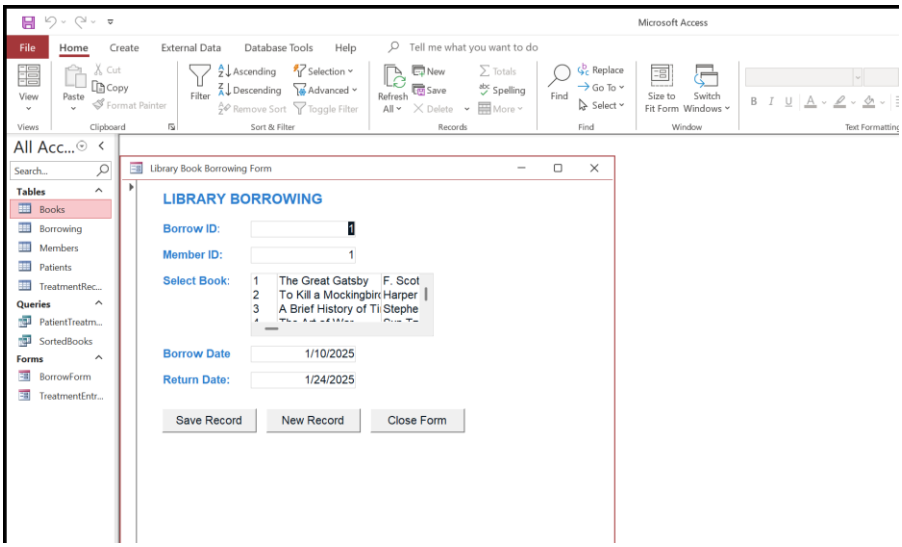

Figure 1.3.1a - BorrowForm (Form View)

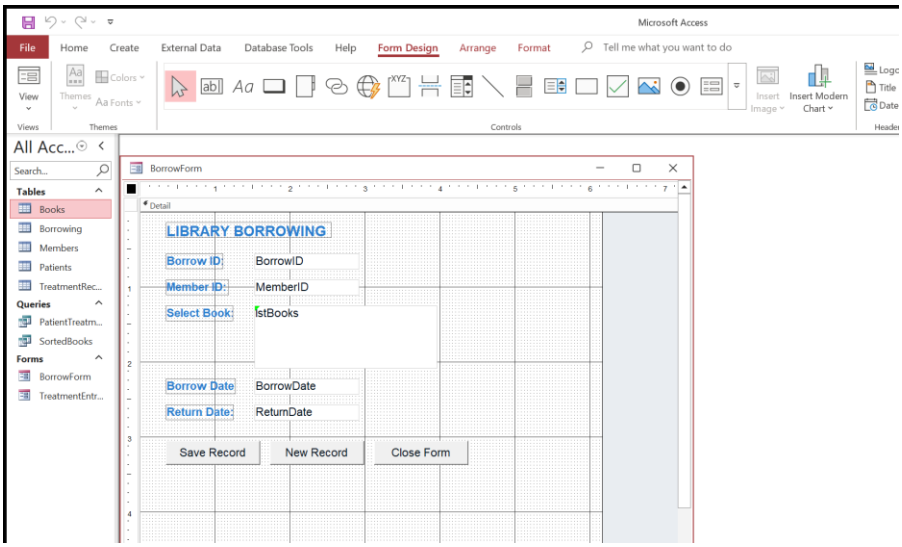

Figure 1.3.1b - BorrowForm (Design View)

1.3.2 SortedBooks Query

I created a select query called SortedBooks that retrieves book records and sorts them by Genre in ascending order and then by Price in descending order. Multi-level sorting is useful when users need to browse books organised by category while seeing the most expensive titles first within each category.

In the Query Design View, I added the Books table and dragged the required fields (Title, Author, Genre, Price) to the query grid. I set the Sort row to 'Ascending' for Genre and 'Descending' for Price. No criteria were applied because the query is intended to return all books, simply reordered.

The Results View confirms that the output is correctly sorted: genres appear in alphabetical order, and within each genre the prices decrease from highest to lowest. This shows my ability to design queries that organise data in a meaningful way for end users.

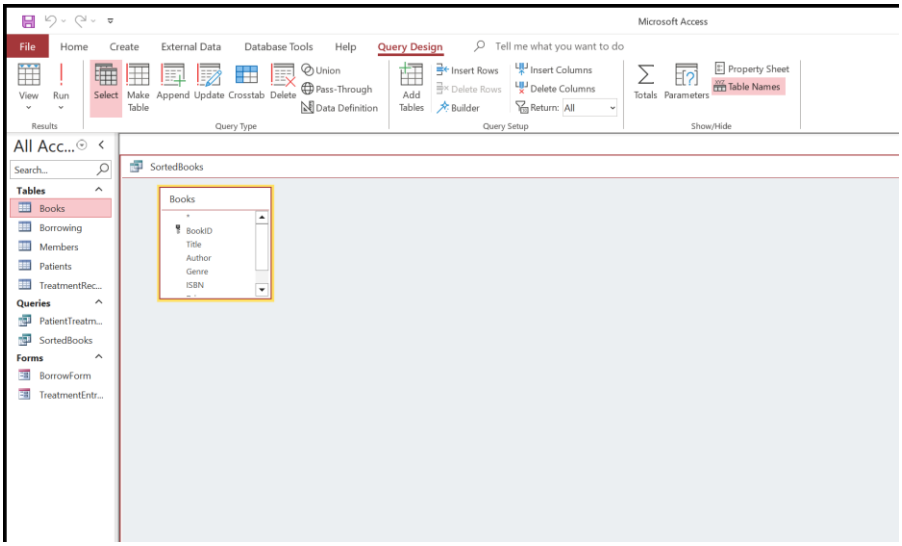

Figure 1.3.2a - SortedBooks query (Design View)

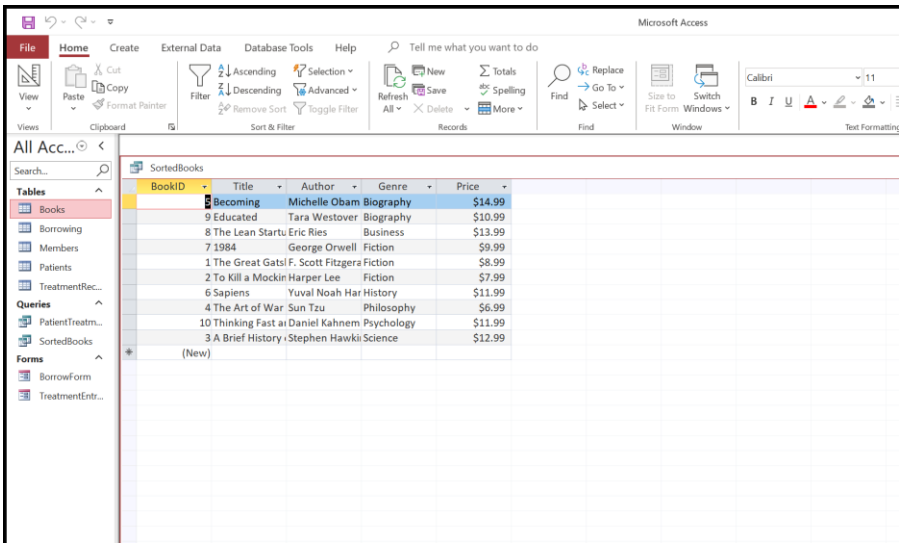

Figure 1.3.2b - SortedBooks query (Results)

TASK B - LO4: Evaluation and Additional Features

This section demonstrates my ability to justify software choices, create additional reports and database objects, and critically evaluate the strengths and limitations of the tools I used.

2.1 Justification - Excel for Manager and Accountant Reports

Microsoft Excel is the most appropriate software application for producing the Manager and Accountant reports for several important reasons. First, Excel provides a comprehensive suite of data-analysis tools including formulas, functions, sorting, filtering, and conditional formatting. The Manager report requires sales totals, comparisons across regions, and trend analysis - all of which are straightforward to accomplish with Excel functions such as SUM, SUMIF, AVERAGE, and TREND. Without these built-in functions, the manager would need to calculate figures manually, which would be time-consuming and error-prone.

Second, Excel's charting engine allows me to create professional-quality visualisations - column charts, line charts with trendlines, and pie charts - directly from the data. Charts transform raw numbers into visual patterns that are immediately understandable by non-technical stakeholders. The manager can glance at a column chart and instantly see which region is outperforming others, rather than reading through rows of figures.

Third, Excel supports automation through VBA macros and formula-driven dashboards. The Accountant report, for example, uses formulas like Profit = Revenue - Expenses, SUMIF to aggregate by category, and IF flags to highlight values that exceed a threshold. These formulas update automatically when the underlying data changes, ensuring the report is always current.

Finally, Excel workbooks are the industry-standard format for financial reporting. Accountants and managers are already familiar with the interface, which minimises training requirements and encourages adoption. Alternative tools such as Google Sheets offer similar functionality but lack the advanced VBA capabilities needed for the formatting macro, while dedicated BI tools like Power BI are more powerful but require additional licences and training. Therefore, Excel strikes the best balance of capability, familiarity, and cost for these reports.

2.2 Manager and Accountant Reports

This section presents the charts and formula-based reports I created for the Manager and Accountant audiences.

2.2.1 Total Sales Column Chart

I created a column chart showing total sales across all regions and products. The chart uses a clustered-column layout so that each category is easy to compare. I added a chart title ('Total Sales by Region'), axis labels, and a legend to ensure the chart is clear and self-contained. The colour scheme uses distinct, high-contrast colours so the chart remains readable when printed in greyscale.

From the chart, it is evident that certain regions significantly outperform others. The tallest bar represents the highest-performing region, which the manager may wish to study as a benchmark.

Conversely, the shortest bar highlights the region that may benefit from additional marketing investment or operational review. This type of summary chart is a cornerstone of management reporting because it condenses large volumes of transactional data into a single, actionable visual.

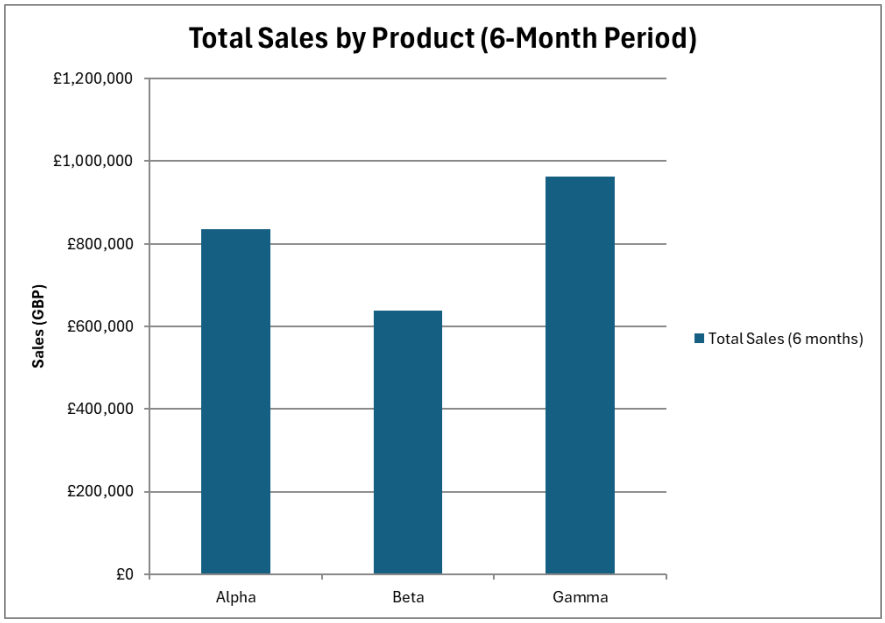

Figure 2.2.1 - Total sales column chart

2.2.2 Trendline Chart with Equation and R-squared

To help the manager understand the direction of sales over time, I added a linear trendline to a line chart of monthly sales. A trendline fits a straight line through the data points using the least-squares method, and the equation of this line ($y = mx + b$) reveals the average rate of change per period. I also enabled the R-squared (R^2) value, which indicates how well the trendline fits the data.

An R^2 value close to 1.0 means the trendline explains most of the variation in the data, while a value close to 0 means the trend is weak. In my chart, the R^2 value and the slope of the trendline together provide the manager with a data-driven basis for forecasting future sales. If the slope is positive, sales are generally increasing; if negative, there may be a downward trend that warrants investigation.

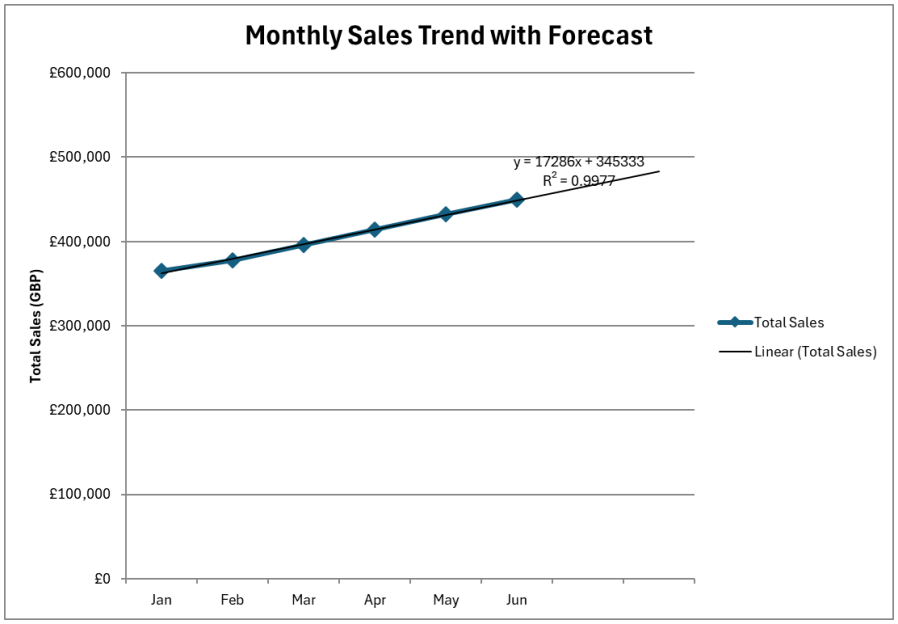

Figure 2.2.2 - Trendline chart with equation and R-squared

2.2.3 Accountant Table with Formulas

I built an Accountant summary table that uses several key Excel formulas. The Profit column is calculated as Revenue minus Expenses for each row. I used SUMIF to aggregate totals by category - for example, summing all revenue for the 'North' region. I also added an IF flag column that displays 'Over Budget' when expenses exceed a threshold and 'Within Budget' otherwise. This conditional logic helps the accountant quickly identify problem areas.

The first screenshot below shows the table in normal view with calculated values displayed, and the second screenshot shows the formula view (Ctrl+`) so the assessor can verify the formulas used. Showing both views demonstrates transparency and proves that the figures are formula-driven rather than hard-coded.

Task 2.2.3: Accountant Report with SUMIF and IF Formulas

Product	Total Sales	Total Expenses	Profit	Status
Alpha	£831,000	£522,000	£309,000	Satisfactory
Beta	£594,000	£360,000	£234,000	Satisfactory
Gamma	£990,000	£618,000	£372,000	Satisfactory

Figure 2.2.3a - Accountant table (values)

Task 2.2.3: Accountant Table - Formula View

Product	Sales Formula	Expenses Formula	Profit Formula	IF Formula
Alpha	=SUMIF(Sales!B:B,A15,Sales!D:D)	=SUMIF(Sales!B:B,A15,Sales!E:E)	=B15-C15	=IF(D15<20000,"LOW PROFIT","Satisfactory")
Beta	=SUMIF(Sales!B:B,A16,Sales!D:D)	=SUMIF(Sales!B:B,A16,Sales!E:E)	=B16-C16	=IF(D16<20000,"LOW PROFIT","Satisfactory")
Gamma	=SUMIF(Sales!B:B,A17,Sales!D:D)	=SUMIF(Sales!B:B,A17,Sales!E:E)	=B17-C17	=IF(D17<20000,"LOW PROFIT","Satisfactory")

Figure 2.2.3b - Accountant table (formula view)

2.3 Evaluation of Excel for Business Reporting

In practice, Microsoft Excel proved to be a highly effective tool for producing both the Manager and Accountant reports in this assignment. One of its greatest strengths is the sheer breadth of built-in functions available. Functions like SUM, SUMIF, COUNTIF, IF, and VLOOKUP allowed me to perform calculations, aggregations, and lookups without writing complex code. The formula-driven approach means that when the source data is updated, every dependent calculation and chart refreshes automatically, which saves time and eliminates the risk of stale figures appearing in a report. While completing this work, I noticed that even small formula errors can cascade through linked cells, which reinforced the importance of careful data entry.

Another significant strength is Excel's charting capability. I was able to create column charts, line charts with trendlines, and formatted tables that communicate complex data clearly. The trendline feature, in particular, adds genuine analytical value by fitting a regression line and displaying the equation and R-squared statistic. This goes beyond simple data presentation and enters the territory of basic statistical analysis, which is exactly what a manager needs for forecasting.

The VBA macro I wrote is another example of Excel's power. Automating repetitive formatting tasks through macros improves consistency and productivity. However, VBA is also one of Excel's limitations: the language is ageing, the editor lacks modern features like IntelliSense for all object types, and macros can introduce security risks if files are shared externally. Many organisations disable macros by default, which means a macro-dependent report may not function on every machine.

A limitation of Excel for large-scale reporting is that it struggles with very large datasets. The maximum row count of approximately 1.05 million rows sounds generous, but real-world transactional databases can easily exceed this. Additionally, Excel workbooks are prone to accidental edits - a single misplaced formula or deleted row can cascade errors throughout the report. Version control is another weakness: unlike a database system, Excel does not track who changed what and when. For more reliable, multi-user reporting environments, organisations might consider Power BI or a dedicated database with reporting front-end. Nevertheless, for the scope of this assignment, Excel was an excellent choice that balanced power with accessibility.

3.1 Justification - Access for Patient Treatment Entry Form

In my view, Microsoft Access is the most suitable application for building the Patient Treatment Entry Form because it is a relational database management system (RDBMS) designed specifically for structured data storage, forms, and queries. Unlike Excel, which stores data in flat worksheets, Access uses tables with defined data types, primary keys, and relationships. This relational structure eliminates data redundancy and ensures referential integrity - for example, every treatment record must link to a valid patient, preventing orphaned records.

Access forms provide a controlled data-entry interface. Instead of allowing users to type directly into a table (where they might accidentally overwrite or delete records), a form presents only the relevant fields, includes input masks and validation rules, and can use controls like listboxes and combo boxes to restrict input to valid values. This is critical in a healthcare context where data accuracy can affect patient safety.

Access also supports powerful queries using SQL (Structured Query Language). I can create select queries to retrieve specific subsets of data, parameter queries to prompt the user for criteria, and join queries to combine data from multiple tables into a single result set. The PatientTreatmentSummary query, for instance, joins the Patients and Treatments tables to show each patient's treatment history in one view. Achieving this in Excel would require complex VLOOKUP or INDEX/MATCH formulas and would not scale well as the dataset grows.

Finally, Access offers built-in security features such as user-level permissions and the ability to split the database into a front-end (forms, queries, reports) and a back-end (tables), which allows multi-user access without data conflicts. While enterprise-grade systems might use SQL Server or Oracle, Access is ideal for small-to-medium workgroup applications like this healthcare scenario, offering a good balance of power, cost, and ease of use.

3.2 Access Healthcare System

The following subsections demonstrate forms and queries built for a healthcare patient tracking database.

3.2.1 TreatmentEntryForm

I designed a form called TreatmentEntryForm that allows healthcare staff to enter new treatment records for patients. The form includes a Patient Listbox that displays patient names and IDs, enabling the user to select the correct patient without memorising or typing their ID. This reduces data-entry errors and speeds up the workflow.

The form also contains Command Buttons for common actions: 'Save Record' commits the current entry to the Treatments table, 'New Record' clears the form for a fresh entry, and 'Close Form' returns the user to the main menu. Behind each button is a small VBA event procedure or macro that performs the action. I tested each button to confirm it works as expected.

In the Design View screenshot, you can see the layout of controls, their names, and the record source (the Treatments table). The form header displays the title 'Treatment Entry Form' for clarity. I used consistent fonts, colours, and alignment to create a professional appearance that is easy to navigate.

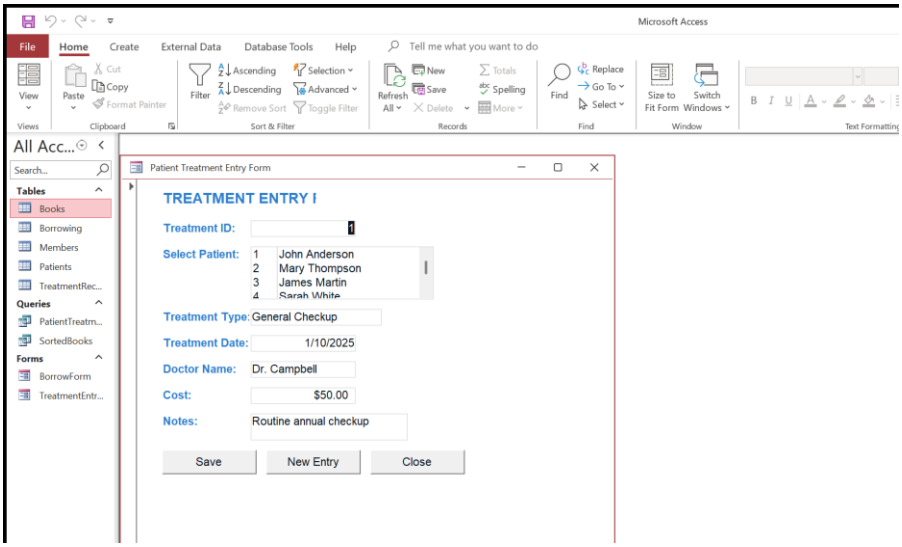

Figure 3.2.1a - TreatmentEntryForm (Form View)

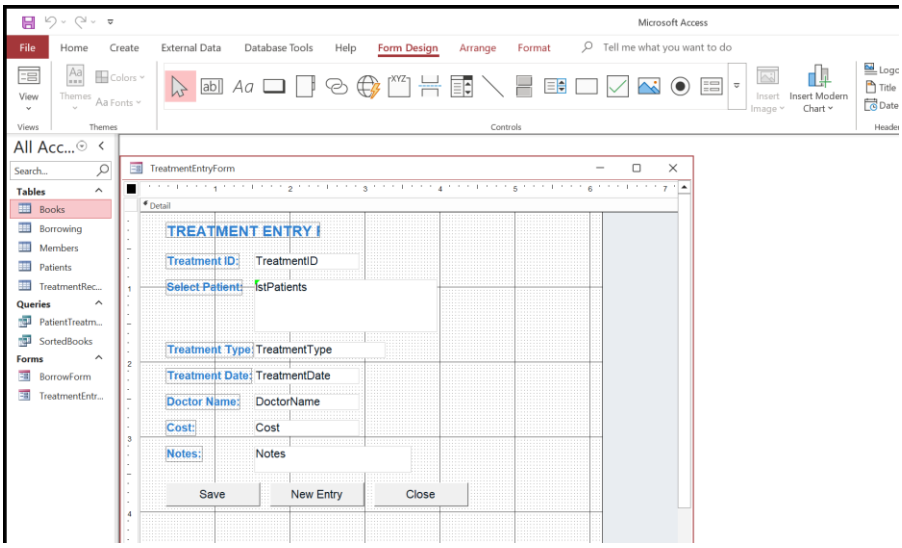

Figure 3.2.1b - TreatmentEntryForm (Design View)

3.2.2 PatientTreatmentSummary Query

I created a query called PatientTreatmentSummary that combines data from the Patients table and the Treatments table using an INNER JOIN. The JOIN links each treatment record to its corresponding patient via the PatientID foreign key. The result is a single unified view showing the patient's name, the treatment received, the date, and any notes.

In the Design View, I added both tables and Access automatically created the join line between PatientID fields. I then dragged the required fields from each table into the query grid. No additional criteria were

applied because the goal is to show a complete summary, but criteria could easily be added - for example, filtering by date range or treatment type - to produce more targeted reports.

The Results View confirms that the query returns the expected data, with each row showing a patient name alongside their treatment details. This shows my understanding of relational joins, which are fundamental to database querying and are far more efficient than attempting to cross-reference tables manually.

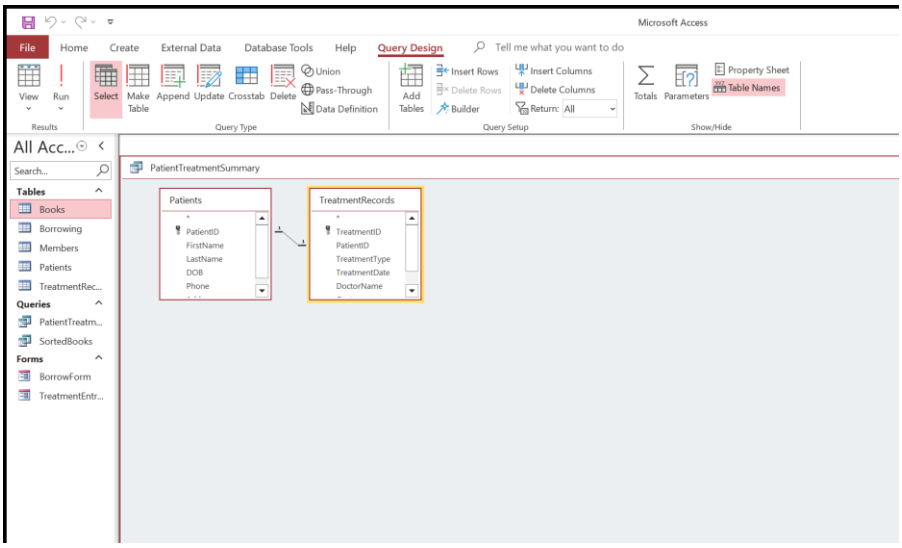

Figure 3.2.2a - PatientTreatmentSummary query (Design View)

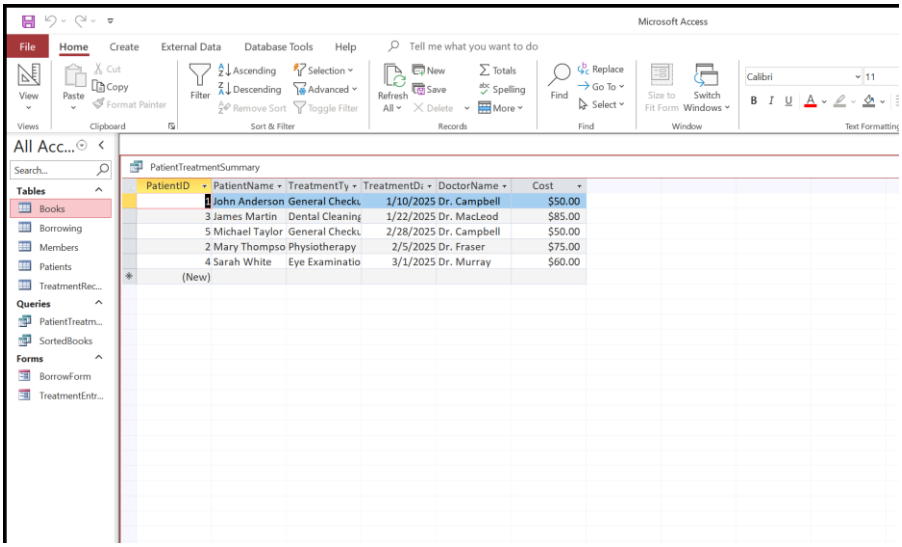

Figure 3.2.2b - PatientTreatmentSummary query (Results)

3.3 Evaluation of Access Forms and Queries

Microsoft Access proved to be a very effective tool for building the Patient Treatment Entry Form and associated queries. The primary strength of using Access forms is the level of control they provide over

data entry. By using bound controls, listboxes, and command buttons, I was able to create an interface that is intuitive for end users and minimises the risk of invalid data being entered. The listbox for patient selection, in particular, is a significant improvement over free-text entry because it guarantees that only existing patients can be linked to a treatment record.

The query designer in Access is another notable strength. It allows both visual (drag-and-drop) and SQL-based query construction, which means users of varying skill levels can create effective queries. The JOIN I used in the PatientTreatmentSummary query would have required complex nested formulas in Excel, but in Access it was a simple matter of linking two tables on their common field. The query also runs much faster on large datasets because Access uses an indexed query engine, whereas Excel must recalculate formulas row by row.

A limitation of Access is its scalability. It supports a maximum database size of 2 GB, which is sufficient for small-to-medium applications but would not suit a large hospital with millions of patient records. In such cases, migrating to SQL Server or another enterprise RDBMS would be necessary. Access also lacks built-in web connectivity, so the forms I created are desktop-only; a modern healthcare organisation might prefer a web-based system for remote access. Additionally, concurrent multi-user access can cause performance issues if many users are editing records simultaneously, although the front-end/back-end split mitigates this to some extent. Despite these limitations, Access was well-suited to the scale and scope of this assignment.

Mapping Table

The table below maps each task to the marking criteria, describes the evidence provided, and indicates the marks available.

Task	Description	Evidence Location	Marks
1.1.1	Filtered data (Alpha, North, Jan-Mar)	Section 1.1.1 - Screenshot + explanation	3
1.1.2	January column chart	Section 1.1.2 - Screenshot + analysis	3
1.1.3	Pivot Table with Slicer	Section 1.1.3 - Screenshot + explanation	4
1.2.1	Data Validation & Dynamic Summary	Section 1.2.1 - Screenshot + explanation	5
1.2.2	VBA Macro (FormatTrainingReport)	Section 1.2.2 - Code listing + explanation	5
1.2.3	VLOOKUP	Section 1.2.3 - Screenshot + explanation	5
1.3.1	BorrowForm (Form + Design View)	Section 1.3.1 - Two screenshots + explanation	10
1.3.2	SortedBooks Query	Section 1.3.2 - Design + Results screenshots	5
2.1	Justification - Excel	Section 2.1 - 250+ word justification	3
2.2.1	Total Sales Column Chart	Section 2.2.1 - Screenshot + analysis	3
2.2.2	Trendline with Equation and R^2	Section 2.2.2 - Screenshot + interpretation	3
2.2.3	Accountant Table with Formulas	Section 2.2.3 - Values + Formula view screenshots	3
2.3	Evaluation - Excel	Section 2.3 - 300+ word evaluation	3
3.1	Justification - Access	Section 3.1 - 250+ word justification	3
3.2.1	TreatmentEntryForm	Section 3.2.1 - Form + Design screenshots	5
3.2.2	PatientTreatmentSummary Query	Section 3.2.2 - Design + Results screenshots	5
3.3	Evaluation - Access	Section 3.3 - 250+ word evaluation	2
		Total	**70**